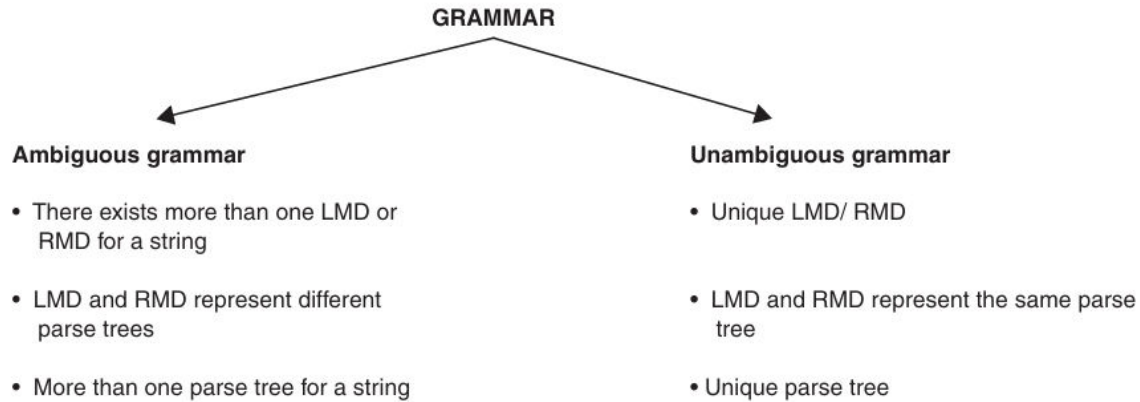


3.9 Ambiguous Grammar

A CFG is ambiguous if there exists more than one parse tree or equivalently, more than one leftmost derivation and one rightmost derivation for at least one word in its CFL.



Remember that there is no algorithm that automatically checks whether a grammar is ambiguous or not. The only way to check ambiguity is “to choose an appropriate input string and by trial and error find the number of parse trees.” If more than one parse tree exists, the grammar is ambiguous.

Example 26:

The grammar $G: S \rightarrow SS \mid a \mid b$ is ambiguous. This means at least some of the strings in its language have more than one leftmost derivation. However, it may be that some strings in the language have only one derivation. Identify the string that has exactly two leftmost derivations in G .

Solution:

A string of length 1 has only one leftmost derivation, for example, $S \Rightarrow a$. A string of length 2 has only one derivation, for example, $S \Rightarrow SS \Rightarrow aS \Rightarrow ab$.

However, a string of length 3 has exactly two derivations, for example, $S \Rightarrow SS \Rightarrow SSS \Rightarrow aSS \Rightarrow abS \Rightarrow aba$ and $S \Rightarrow SS \Rightarrow aS \Rightarrow aSS \Rightarrow abS \Rightarrow aba$. In general, we can decide whether the first S generates a single terminal or two S 's.

On the other hand, strings of length four or more have more than two derivations. We can either start $S \Rightarrow SS \Rightarrow SSS \Rightarrow SSSS$ or $S \Rightarrow SS \Rightarrow SSS \Rightarrow aSS \Rightarrow aSSS$ or $S \Rightarrow SS \Rightarrow aS \Rightarrow aSS$, and there are other options as well.

Hence, strings like “aaa,” “bbb,” “aba,” “bab,” etc., has only two derivations.

There is no algorithm that converts an ambiguous grammar to its equivalent unambiguous grammar.

Example 25:

Show that the following grammar is ambiguous.

$$\begin{aligned} E &\rightarrow id \mid E + E \\ &\mid E * E \mid E - E \end{aligned}$$

Solution:

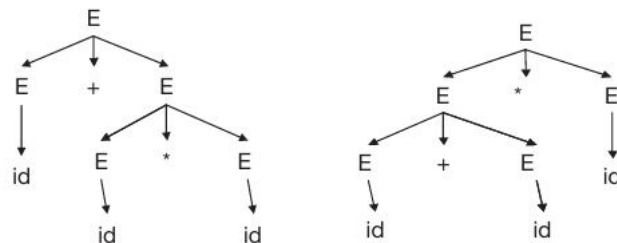
LMD: for string $id + id * id$ is

$$\begin{aligned} E &\Rightarrow \underline{E} + E \\ &\Rightarrow id + \underline{E} \\ &\Rightarrow id + \underline{E} * E \\ &\Rightarrow id + id * \underline{E} \\ &\Rightarrow id + id * id \end{aligned}$$

RMD: for string $id + id * id$ is

$$\begin{aligned} E &\Rightarrow E * \underline{E} \\ &\Rightarrow \underline{E} * id \\ &\Rightarrow E + \underline{E} * id \\ &\Rightarrow \underline{E} + id * id \\ &\Rightarrow id + id * id \end{aligned}$$

Parse trees represented by above LMD and RMD are as follows:



Example 27:

Consider the grammar 'G' with

terminals: a, b

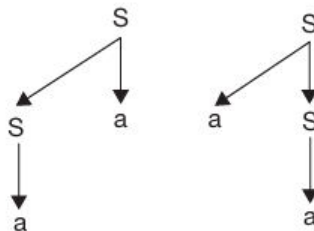
nonterminals: S

productions: $S \rightarrow aS \mid Sa \mid a$.

Show that G is ambiguous.

Solution:

The word "aa" can be generated by two different trees:



Therefore, this grammar is ambiguous.

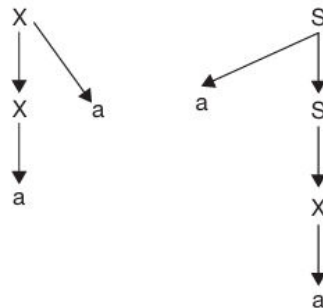
Example 28:

Consider the grammar "G" with
 terminals: a, b
 nonterminals: S, X
 productions: $S \rightarrow aS \mid aSb \mid X$
 $X \rightarrow Xa \mid a$
 Show that G is ambiguous.

Solution:

The word "aa" has two different derivations that correspond to different syntax trees:

$$S \Rightarrow X \Rightarrow Xa \Rightarrow aa$$

$$S \Rightarrow aS \Rightarrow aX \Rightarrow aa$$


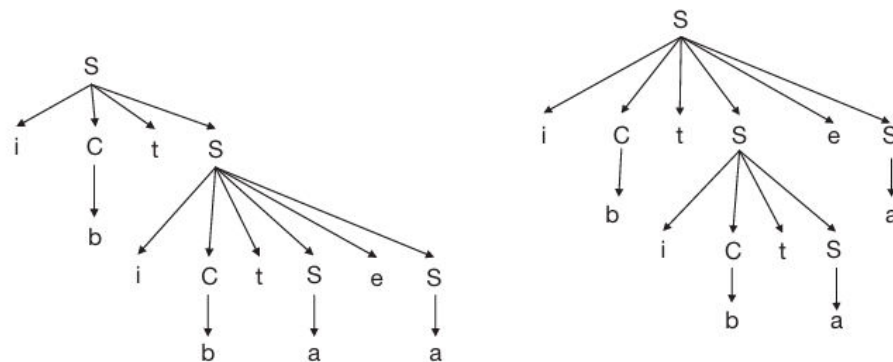
Hence, the grammar is ambiguous.

Example 30:

Check whether the given grammar is ambiguous or not.
 $S \rightarrow i C t S \mid i C t S e S \mid a$
 $C \rightarrow b$

Solution:

To check the ambiguities take an input string. Let us say the input string is "ibtibtaea."
 Let us draw the derivation trees.



Hence, grammar is ambiguous.

Example:

Convert the following grammar to equivalent unambiguous grammar.

$$R \rightarrow R + R \mid RR \mid R^* \mid a \mid b$$

Solution:

This is the grammar that defines regular expressions with basic operations union, concatenation, and Kleenes closure. Here, priority of operators is the highest for closure, next highest for concatenation, and least or union.

So start defining least to highest. So the resulting grammar is

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T F \mid F$$

$$F \rightarrow F^* \mid a \mid b$$

So given a grammar, by using the above procedure we can get precedence and associativity of operators. For example,

$$A \rightarrow A \# B \mid B$$

$$B \rightarrow C \$ B \mid C$$

$$C \rightarrow C @ D \mid D$$

$$D \rightarrow a \mid b$$

Here “#,” “\$,” and “@” are “unknown” binary operators. From the grammar, we can get precedence and associativity of operators as follows:

“@” has the highest precedence and is left associative

“\$” has the next highest precedence and is right associative

“#” has the least precedence and is left associative

9. Check whether the following grammar is ambiguous for $w = ab$

$$S \rightarrow aB \mid ab$$

$$A \rightarrow aAB \mid a$$

$$B \rightarrow ABb \mid b$$

Solution:

The string “ab” can be derived using LMD as follows:

$$1. S \Rightarrow ab$$

$$2. S \Rightarrow a\underline{B} \Rightarrow ab$$

Since there are two possible ways to derive the string, it is ambiguous for w .

10. Check whether the following grammar is ambiguous for $w = aab$.

$$S \rightarrow AB \mid aaB$$

$$A \rightarrow a \mid Aa$$

$$B \rightarrow b$$

Solution:

The string “aab” can be derived using LMD as follows:

$$1. S \Rightarrow AB \Rightarrow AaB \Rightarrow aaB \Rightarrow aab$$

$$2. S \Rightarrow aa\underline{B} \Rightarrow aab$$

Since there are two possible ways to derive the string, it is ambiguous for w .

Example 1:

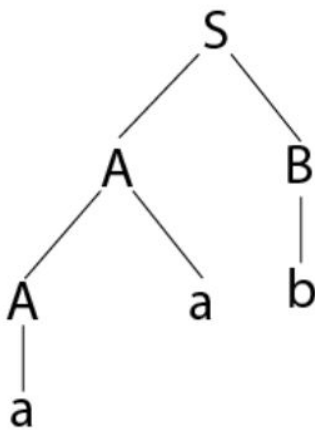
Consider a grammar G is given as follows:

```
S → AB | aaB
A → a | Aa
B → b
```

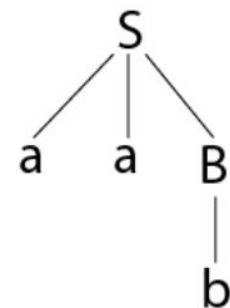
Determine whether the grammar G is ambiguous or not. If G is ambiguous, construct an unambiguous grammar equivalent to G .

Solution:

Let us derive the string "aab"



Parse tree 1



Parse tree 2

As there are two different parse tree for deriving the same string, the given grammar is ambiguous.

Unambiguous grammar will be:

```
S → AB
A → Aa | a
B → b
```

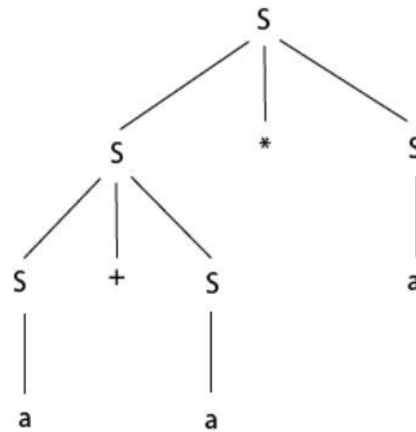
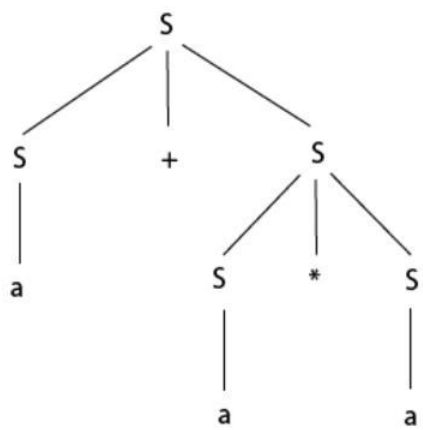
Example 4:

Check that the given grammar is ambiguous or not. Also, find an equivalent unambiguous grammar.

```
S → S + S  
S → S * S  
S → S ^ S  
S → a
```

Solution:

The given grammar is ambiguous because the derivation of string aab can be represented by the following string:



Unambiguous grammar will be:

```
S → S + A |  
A → A * B | B  
B → C ^ B | C  
C → a
```